Chapter 3. Linked Lists

Instructor: Kuen-Liang Sue
IM/NCU

Outline

- 3-1 Linear List Concepts
- 3-2 Linked List Concepts
- 3-3 Linked List Algorithms
- 3-4 Processing a Linked List
- 3-5 List Applications
- 3-6 Complex Linked List Structures

Preface

- Arrays
  - Arrays are easy to create and use
  - But they are inefficient when
    - sequenced data need to be inserted or deleted
- Linked list
  - Linked list efficiently handles insertion and deletion
  - it is inefficient for search and retrieval

3-1 Linear List Concepts

- Linear List
  - The simplest linear list structure, the array, is found in virtually all programming

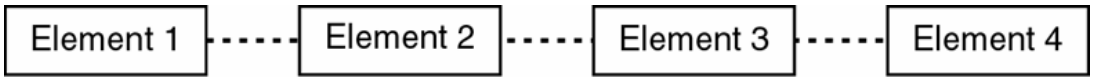


Figure 3-1 A linear list

  - Linear list can be divided into two categories
    - General
    - Restricted

Linear List Concepts

- General list
  - Data can be inserted and deleted anywhere
  - No restrictions on the processing operations
  - Random list
    - There is no ordering of the data
  - Ordered list
    - The data are arranged according to a key
    - A key is one or more field within a structure that are used to identify the data or otherwise control their use

Linear List Concepts

- Restricted list
  - Data can only be added or deleted at the ends of the structure
  - Processing is restricted to operate on the data at the ends of the list
  - First in-first out (FIFO): queue
  - Last in-first out (LIFO): stack

Linear List Concepts

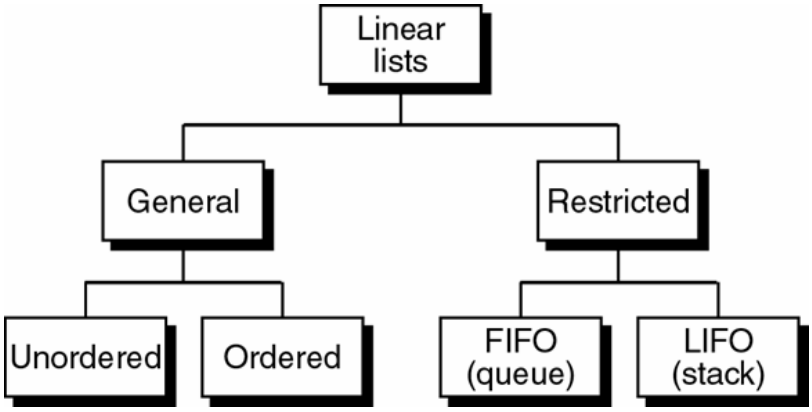


Figure 3-2 Types of lists

Linear List Concepts

- Four operations associated with linear list
  - Insertion
  - Deletion
  - Retrieval
    - The first three apply to all lists
  - List traversal
    - not applicable to restricted lists

Insertion

- Random list Insertion
  - Be made
    - at the beginning
    - in the middle
    - at the end of the list
  - Depending on the type of general list
  - Few computer applications use random lists

Insertion

- Ordered list insertion
  - The ordering of the list must be maintained

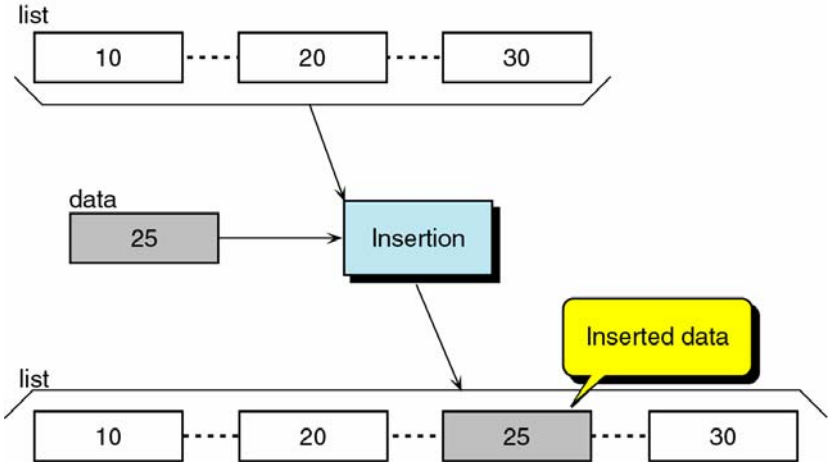


Figure 3-3 Ordered list insertion

Deletion

- Steps for deletion
  - Search the list to locate the data being deleted
    - Any sequential search can be used to locate the data
  - After deletion from an array
    - the data following the deleted item must be shifted
      - to replace the empty element
    - linked list can eliminate this shifting of data

Deletion

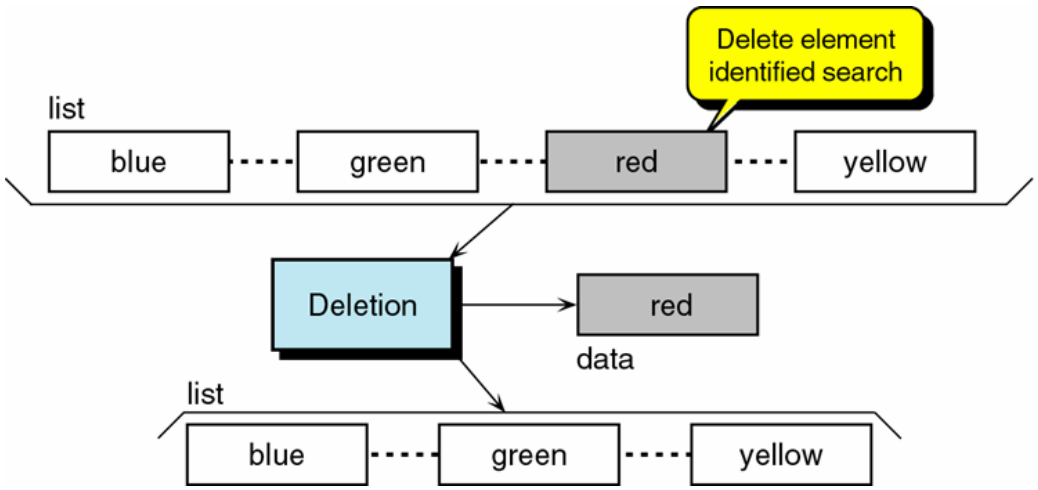


Figure 3-4 General list deletion

Retrieval

- Steps for list retrieval
  - Data is located in a list
  - Presented to the calling module
  - Without changing the contents of the list
    - Any sequential search can be used to locate the data
    - Retrieval from a restricted list depends on the particular list being used

Retrieval

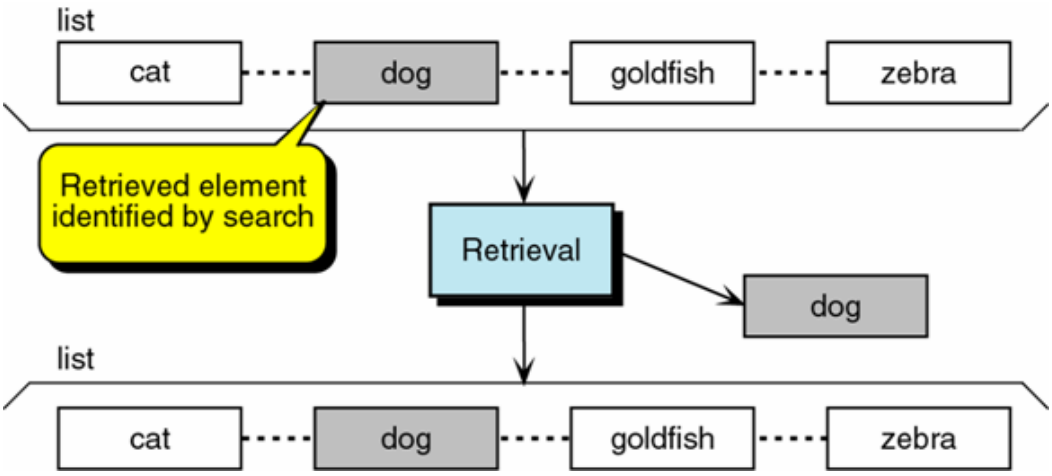


Figure 3-5 Linear list retrieval

Traversal

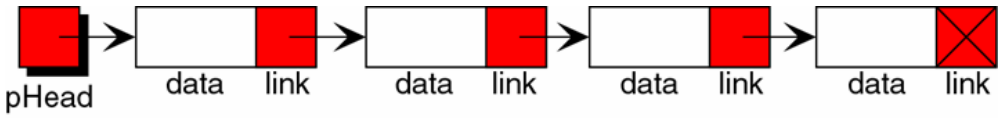
- List traversal
  - A special case of retrieval
    - All elements are retrieved in sequence
  - Requires a looping algorithm rather than a search
    - Each execution of the loop processes one element
    - Loop terminates when all elements have been processed
- Other processes can be used with a linear list
  - sorting the list
  - totaling the value

3-2 Linked List Concepts

- Linked list
  - An ordered collection of data
  - Each element contains the location of next element
  - Each element contain two parts: **data** and **link**
    - The data part holds the useful information
    - The link is used to chain the data together
  - Singly linked list
    - It contains only one link to a single successor

Linked List Concepts

- Advantage and limit of the linked list
  - Easily inserted and deleted
  - Cannot use a binary search
- Empty linked list
  - A null head pointer



(a) A linked list with a head pointer: pHead



(b) An empty linked list

Figure 3-6 A linked list

Linked List Concepts

- Node
  - Elements in a linked list are called nodes
  - A node has at least two field
    - One contains the data
    - The other is the address of the next node in the sequence
  - Figure 3-7 shows three different node structure
  - Self-referential structures
    - Nodes in a linked list are called Self-referential structures
      - Each instance of the structure contains a pointer to another instance of the same structural type

Linked List Concepts

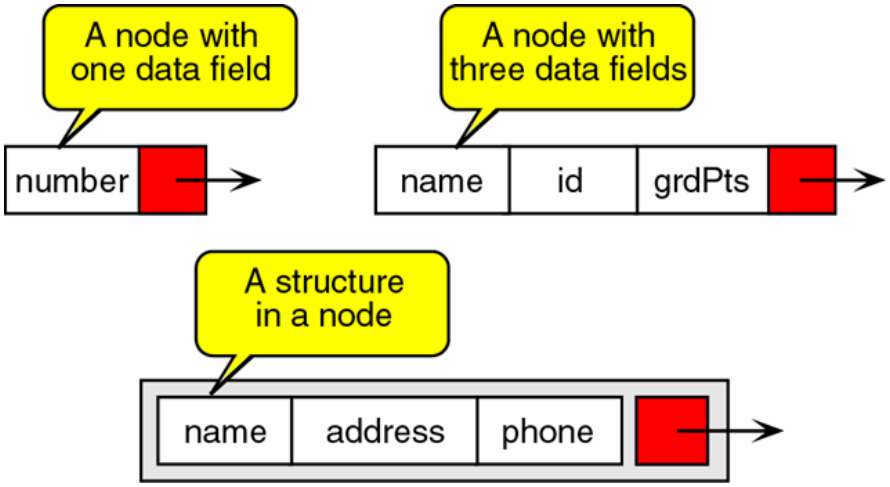


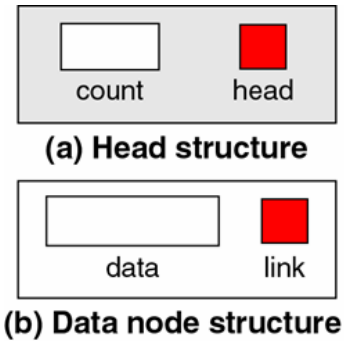
Figure 3-7 Nodes

Linked List Data Structure

- Feature of a linked list
  - There is no physical relationship between the nodes
    - That is, they are not stored contiguously
- Header pointer
  - A pointers to distinguish the beginning of the list
  - To identify the first logical node in the list
  - Called **pHead**

Linked List Data Structure

- Head node structure
  - A structure that stores
    - The head pointer
    - Metadata
      - Data about the list




```
list
   count    <integer>
   head     <pointer>
end list

node
   data     <dataType>
   link     <pointer>
end node
```


Figure 3-8 Linked list node structure

Linked List Data Structure

- Data node structure
  - A typical data type
    - A **key** field for searching by key

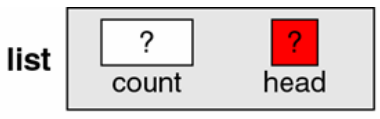
```
dataType
    key     <keyType>
    field1    <...>
    field2    <...>
       ...
    fieldN    <...>
end dataType
```

Linked List Data Structure

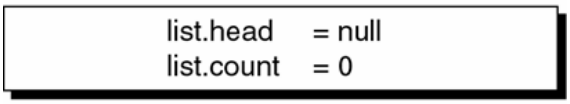
- Pointers to linked lists
  - pLoc
    - Pointer to the current node you are accessing
  - pLast (pRear)
    - Pointer to the last node in the list
    - Programming is more efficient
      - if there is a pointer to the last node

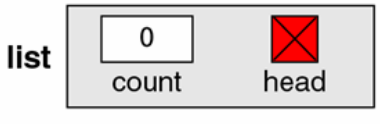
3-3 Linked List Algorithms

- Create list
  - receives the head structure and
  - initializes the metadata



(a) Before create





(b) After create

Figure 3-9 Create list

Linked List Algorithms

```
algorithm createList (ref list <metadata>)
Initializes metadata for a linked list.
    Pre     list is metadata structure passed by reference
    Post    metadata initialized
 1  list.head  = null
 2  list.count = 0
 3  return
end createList
```

Algorithm 3-1 Create linked list

Insert Node

- Insert node
  - Insert node adds data to a linked list
  - Three steps to the insertion
    - Allocate memory for the new node and insert data
    - Point the new node to its successor
    - Point the new node's predecessor to the new node
  - Two states of the node that precedes the new node
    - The predecessor is null or not null

Insert Node

- Insert into empty list
  - Set the link of the new node to a null pointer
  - Assign the list head pointer the address of new node

```
pNew->link  = list.head        Set link to null pointer
list.head   = pNew             Point list to first node
```

  - Note the order of these two statement
    - Must first point the new node to its successor
    - Then change the head pointer

Insert into Empty List

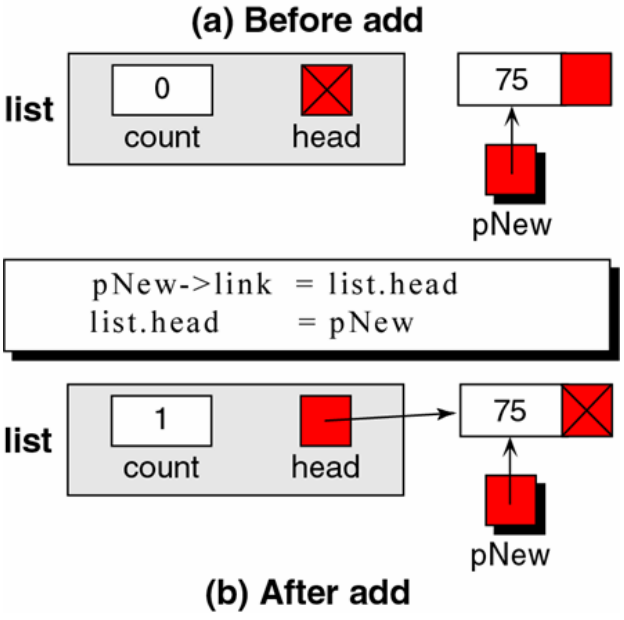


Figure 3-10 Add node to empty node

Insert Node

- Insert at beginning
  - Insert a node before the first node of the list
  - Point the new node to the first node of the list
  - Set the head pointer to point to the new first node

```
pNew->link     = list.head
list.head      = pNew
```

  - Logically, inserting into an empty list is the same as inserting at the beginning of a list

Add Node at Beginning

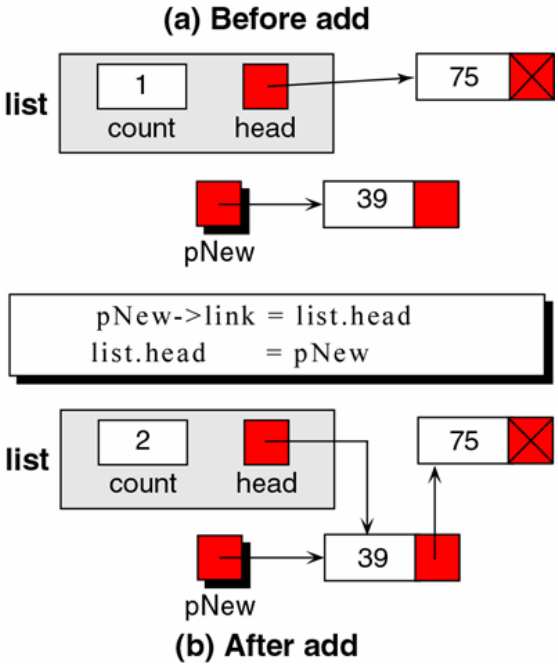


Figure 3-11 Add node at beginning

Insert Node

- Insert in middle
  - To insert a node between two nodes
    - we point the new node to its successor
    - Then point its predecessor to the new node
  - The address of the new node's successor can be found in the predecessor's linked field

```
pNew->link    = pPre->link
pPre->link    = pNew
```


Add Node in Middle

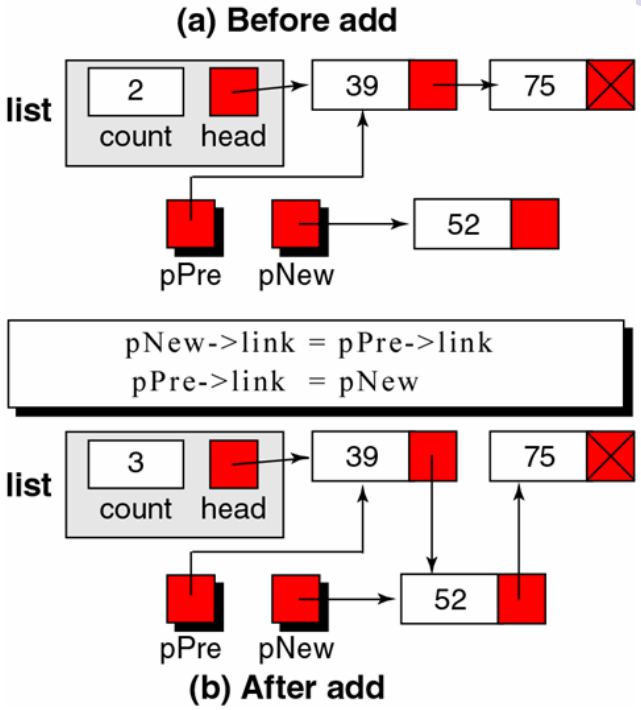


Figure 3-12 Add node in middle

Insert Node

- Insert at end
  - Only point the predecessor to the new node

```
pNew->link    = null pointer
pPre->link    = pNew
```

  - Revised code
    - The code becomes exactly the same as the code for inserting in the middle of the list

```
pNew->link    = pPre->link
pPre->link    = pNew
```


Add Node at End

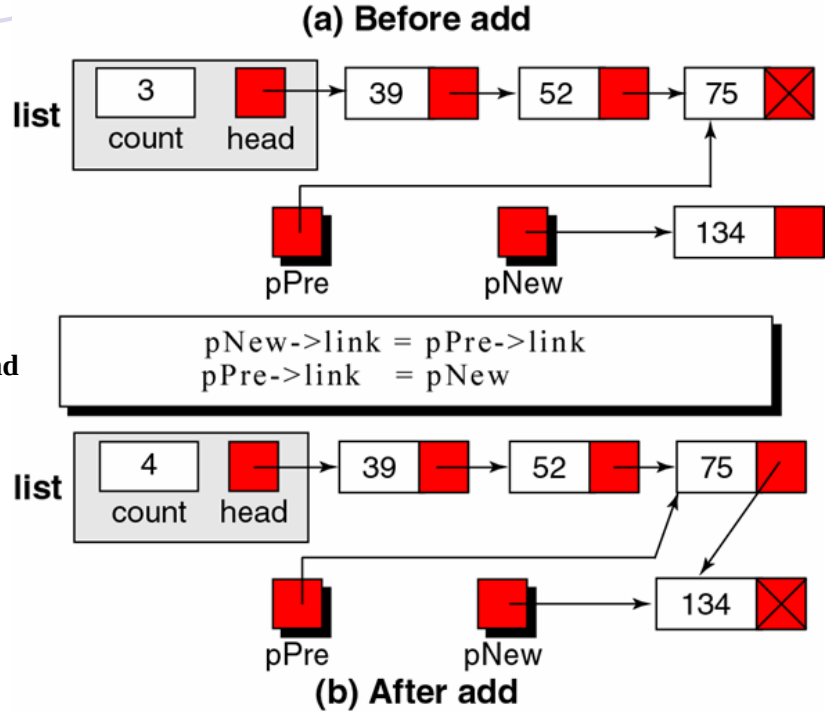


Figure 3-13 Add node at end

Insert Node Algorithm

```
algorithm insertNode (ref list     <metadata>,
                      val pPre     <node pointer>,
                      val dataIn   <dataType>)
Inserts data into a new node in the linked list.
   Pre      list is metadata structure to a valid list
            pPre is pointer to data's logical predecessor
            dataIn contains data to be inserted
   Post     data have been inserted in sequence
   Return   true if successful, false if memory overflow
 1 allocate (pNew)
 2 if (memory overflow)
   1  return false
 3 end if
 4 pNew->data = dataIn
 5 if (pPre null)
   Adding before first node or to empty list.
   1  pNew->link  = list.head
   2  list.head   = pNew
```

Algorithm 3-2 Insert linked list node

Insert Node Algorithm

```
 6 else
     Adding in middle or at end.
   1  pNew->link  = pPre->link
   2  pPre->link  = pNew
 7 end if
 8 list.count = list.count + 1
 9 return true
end insertNode
```

Algorithm 3-2 Insert linked list node *(continued)*

- Algorithm 3-2 analysis
  - Overflow
    - When memory is exhausted, the insert is in an overflow state
    - Let the calling module decide whether it needs to abort or whether some other action is appropriate

Delete Node

- Delete node
  - logically removes a node from the linked list
    - by changing various link pointers
- Steps to delete Node
  - locate the node to be deleted
  - change its predecessor's link to point to its successor
  - physically deleting the node from dynamic memory

Delete Node

- Delete situations
  - The only node
  - The first node
  - A node in the middle of the list
  - The last node of a list
- Four situations reduce to only two combinations
  - Delete the first node
  - Delete any other node
- In all cases, the node to be deleted is identified by a pointer (**pLoc**)

Delete Node

- Delete first node
  - If the predecessor is a null pointer
    - we are deleting the first node
  - Steps
    - reset the head pointer to point to the first node's successor
    - recycle the memory for the delete node

```
list.head = pLoc->link
recycle (pLoc)
```

  - It also applies when deleting the only node
    - Set the head pointer to a null pointer

Delete First Node

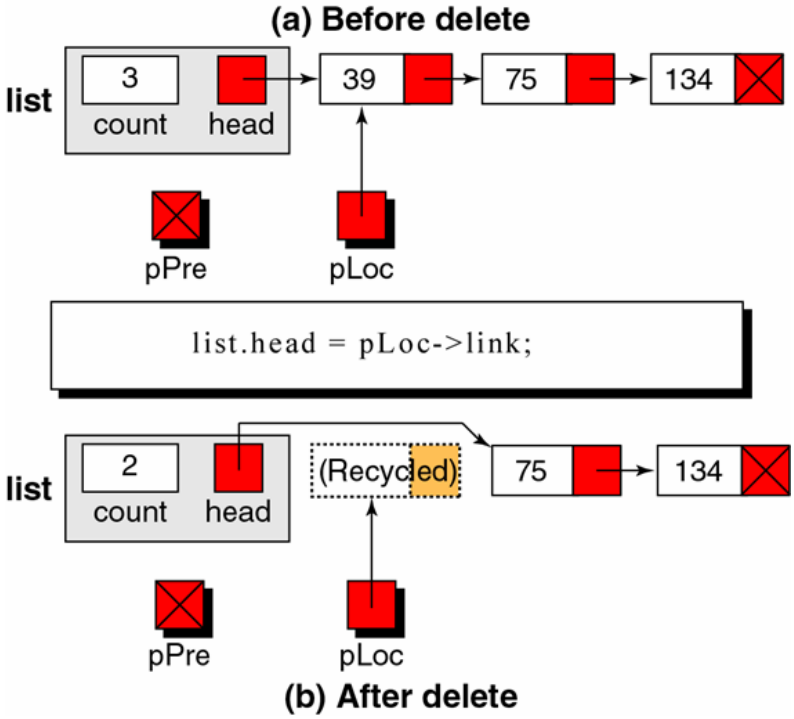


Figure 3-14 Delete first node

Delete Node

- General delete case
  - The same logic applies to delete any node
    - either in the middle of list
    - or at the end of the list
  - How to do
    - Point the predecessor to the successor of the node being deleted

```
pPre->link = pLoc->link
recycle (pLoc)
```


General Delete Case

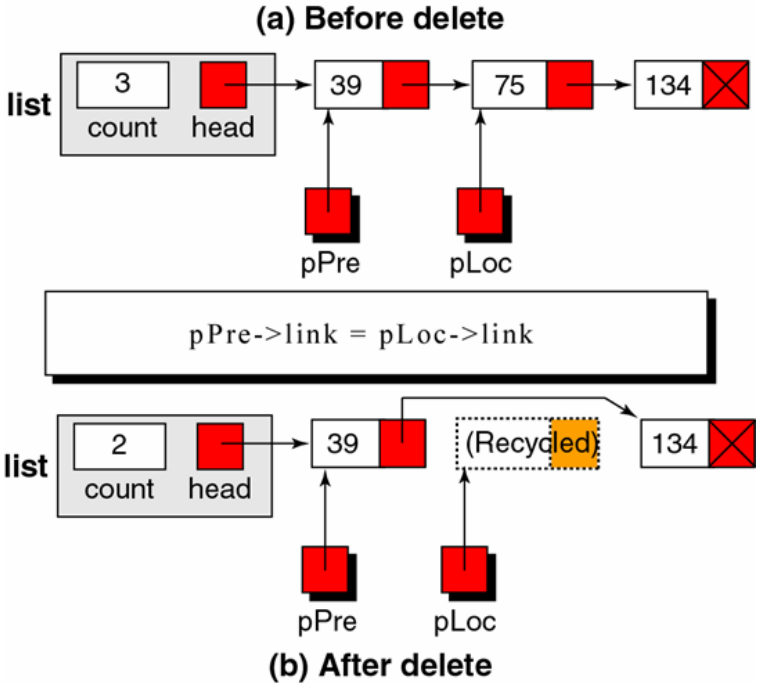


Figure 3-15 Linked list delete general case

Delete Node

- Delete node algorithm

```
algorithm deleteNode  (ref list      <metadata>,
                       val pPre      <node pointer>,
                       val pLoc      <node pointer>,
                       ref dataOut   <dataType>)
Deletes data from a linked list and returns it to calling module.
    Pre     list is metadata structure to a valid list
            pPre is a pointer to predecessor node
            pLoc is a pointer to node to be deleted
            dataOut is variable to received deleted data
    Post    data have been deleted and returned to caller
```

Algorithm 3-3 Linked list delete node

Delete Node

```
 1  dataOut = pLoc->data
 2  if (pPre null)
      Deleting first node
      1   list.head = pLoc->link
 3  else
      Deleting other nodes
      1   pPre->link = pLoc->link
 4  end if
 5  list.count = list.count - 1
 6  recycle (pLoc)
 7  return
end deleteNode
```

Algorithm 3-3 Linked list delete node *(continued)*

Delete Node

- Algorithm 3-3 analysis
  - The node to be deleted must be identified
    - before the procedure's called
  - The *recycle* statement is moved out of the selection logic
    - The same statement appear in both true and false blocks

Search List

- Search list
  - is used by several algorithms to locate data in a list
    - e.g. Insert data, delete data, retrieve data
  - Use a sequential search
    - Because no physical relationship among the nodes
    - When data are stored in an array, we use a binary search
  - Suppose the list is ordered
    - What information will be returned ?
      - the location of the element when it is found
      - the location where it should be placed when it is not found

Search List

- Ordered list search
  - Sequential search in ordered table
- To search a list on a key, we need a key field

```
dataType
    key     <keyType>
    field1    <...>
    field2    <...>
        ...
    fieldN    <...>
end dataType
```

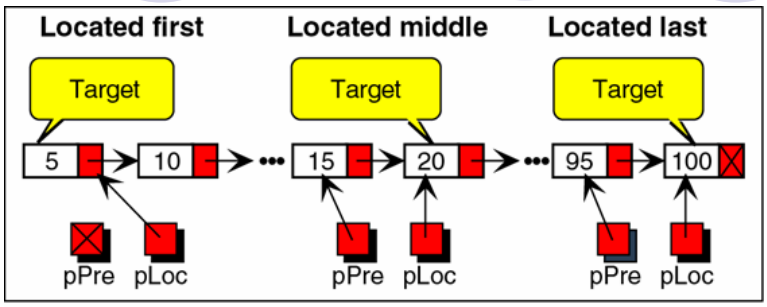

Search List

- Given a target key
  - attempt to locate the requested node by comparison

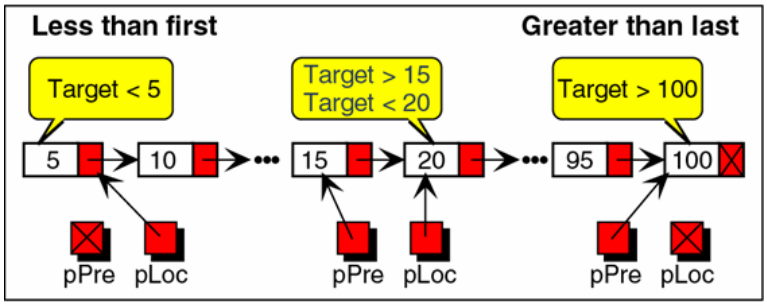
Condition	pPre	pLoc	Return
Target < first node	Null	First node	False
Target equal first node	Null	First node	True
First < target < last	Largest node < target	First node > target	False
Target equal middle node	Node's predecessor	Equal node	True
Target equal last node	Last's predecessor	Last node	True
Target > last node	Last node	Null	False

Table 3-1 List search results

Search List



(a) Successful searches (return true)



(b) Unsuccessful searches (return false)

Search List

```
algorithm searchList (val list     <metadata>,
                      ref pPre     <node pointer>,
                      ref pLoc     <node pointer>
                      val target   <key type>)
Searches list and passes back address of node containing target
and its logical predecessor.
    Pre     list is metadata structure to a valid list
            pPre is pointer variable for predecessor
            pLoc is pointer variable for current node
            target is the key being sought
    Post    pLoc points to first node with equal/greater key
            -or- null if target > key of last node
            pPre points to largest node smaller than key
            -or- null if target < key of first node
    Return  true if found, false if not found
 1  pPre = null
 2  pLoc = list.head
 3  loop (pLoc not null AND target > pLoc->data.key)
    1   pPre = pLoc
```

Algorithm 3-4 Search linked list

Search List

```
      2   pLoc = pLoc->link
   4  end loop
      Set return value
   5  if (pLoc null)
      1   found = false
   6  else
      1   if (target equal pLoc->data.key)
          1   found = true
      2   else
          1   found = false
      3   end if
   7  end if
   8  return found
end searchList
```

Algorithm 3-4 Search linked list *(continued)*

Search List

- Algorithm 3-4 analysis
  - Examine the loop and note that there are two tests
    - The first test protects us from running off the end of the list
    - The second test stops the loop
      - when we find the target or, if the target doesn't exit
  - The search will be more efficient if has a rear pointer
    - A rear pointer is a metadata field
      - that contains the address of the last node
    - We can test the last node to make sure that
      - the target wasn't larger than its key value

Unordered List Search

- Unordered list search
  - It is often necessary to search on a list attribute
    - rather than the key
    - It means the list is unordered according to the attribute
  - To search a list on any field other than the key
    - Use a simple sequential search
  - The problem with nonkey searches is that
    - multiple elements often satisfy the search criteria
    - solution: return a list of all elements that satisfy the criteria

Retrieve Node

- Retrieve node
  - Use search node to locate the data
  - If data are found
    - Move the data to the output area in calling module
    - Return true
  - If not found
    - return false

Retrieve Node

```
algorithm retrieveNode (val list    <metadata>,
                        val key     <key type>,
                        ref dataOut <dataType>)
Retrieves data from a linked list.
    Pre    list is metadata structure to a valid list
           key is target of data to be retrieved
           dataOut is variable to receive retrieved data
    Post   data placed in dataOut
            -or- error returned if not found
    Return true if successful, false if data not found

 1  found = searchList (list, pPre, pLoc, key)
 2  if (found)
    1   dataOut = pLoc->data
 3  end if
 4  return found
end retrieveNode
```

Algorithm 3-5 Retrieve linked list node

Empty List

- Empty list
  - A simple module that returns a Boolean indicating
    - that there are data in the list or that it is empty

```
algorithm emtpyList (val list <metadata>)
Returns Boolean indicating whether the list is empty.
    Pre    list is metadata structure to a valid list
    Return true if list empty, false if list contains data
 1  return (list.count equal to zero)
end emptyList
```

Algorithm 3-6 Empty linked list

- Why write a module when contains only one statement?
  - Hint: Data structure is not necessarily known

Full List

- Full list
  - The only way we can test for a full list is to allocate memory

```
algorithm fullList (val list  <metadata>)
Returns Boolean indicating whether or not the list is full.
    Pre     list is metadata structure to a valid list
    Return  false if room for new node; true if memory full
 1  allocate (pNew)
 2  if (allocation successful)
    1   recycle (pNew)
    2   return false
 3  end if
 4  return true
end fullList
```

Algorithm 3-7 Full linked list

List Count

- List count
  - A simple, one-line module
  - It is necessary because the calling module has no direct access to the list structure

```
algorithm listCount (val list <metadata>)
Returns integer representing number of nodes in list.
    Pre     list is metadata structure to a valid list
    Return  count for number of nodes in list
 1  return (list.count)
end listCount
```

Algorithm 3-8 Linked list count

Traverse List

- Traverse
  - Algorithms that traverse a list start at the first node
    - examine each node in succession
    - until the last node has been processed
  - Usages
    - Change a value in each node
    - Print the list
    - Sum a field in the list
    - Calculate the average of a field

Traverse List

- To traverse the list we need a walking pointer
  - A pointer that moves from node to node
  - Point to the node being processed
- Steps to traverse a list
  - set the walking pointer to the first node
  - use a loop until all of the data have been processed

```
pWalker = list.head
loop (pWalker not null)
    process (pWalker->data)
    pWalker = pWalker->link
```

Traverse List

- Two possible approaches to do the traverse list
  - First approach
    - user controls the loop
    - calling traverse to get the next element
  - Second approach
    - the traverse module controls the loop
    - calling a user-supplied algorithm to process the data
  - We implement the first option
- Add a current position pointer to the head structure
  - because we need to remember where we are in the list

Traverse List

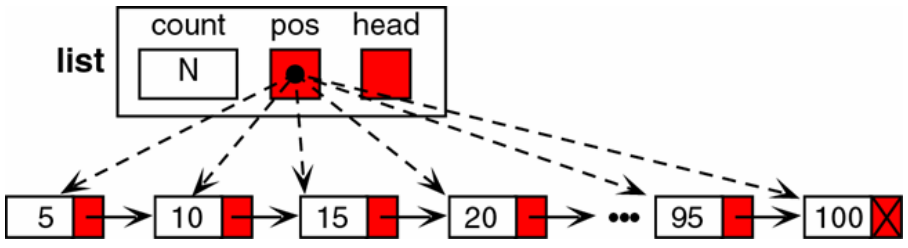


Figure 3-17 Linked list traversal

Traverse List

```
algorithm getNext  (ref list      <metadata>,
                    val fromWhere <Boolean>,
                    ref dataOut   <dataType>)
Traverses a linked list. Each call returns the location of an
element in the list.
    Pre     list is metadata structure to a valid list
            fromWhere is 0 to start at the first element
            dataOut is variable to receive data
    Post    dataOut contains data and true returned
            -or- if end of list, returns false
    Return  true if next element located, false if end of list
```

Algorithm 3-9 Traverse linked list

Traverse List

```
1  if (fromWhere is 0)
   Start from first
   1   if (list.count is zero)
       1   success = false
   2   else
       1   list.pos  = list.head
       2   dataOut   = list.pos->data)
       3   success   = true
2  else
   Continue from pos
   1   if (list.pos->link null)
       End of List
       1   success = false
   2   else
       1   list.pos = list.pos->link
       2   dataOut  = list.pos->data)
       3   success  = true
   3   end if
3  end if
4  return success
end getNext
```

Algorithm 3-9 Traverse linked list *(continued)*

Traverse List

- Algorithm 3-9 analysis
  - Statement 1
    - handle the processing as we start from beginning of the list
    - Statement 1.1
      - We must guard against the traversal of a null list
  - Statement 2
    - handle the processing as we continue from current location
    - Statement 2.1
      - Ensure that there are more data
        - by checking the current position's link field

Destroy List

- Destroy list
  - The list should be destroyed
    - when a list is no longer needed
  - How to destroy list
    - Delete any node still in the list
    - Recycle their memory
    - Set the metadata to a null list condition

Destroy List

```
algorithm destroyList (ref pList <metadata>)
Deletes all data in list.
    Pre     list is metadata structure to a valid list
    Post    All data deleted
 1  loop (list.count not zero)
    1   dltPtr      = list.head
    2   list.head   = dltPtr->link
    3   list.count = list.count - 1
    4   recycle (dltPtr)
 2  end loop
    No data left in list. Reset metadata.
 3  list.pos = null
 4  return
end destroyList
```

Algorithm 3-10 Destroy linked list

3-4 Processing a Linked List

- An integrated program shown in Fig. 3-18
  - A menu module that combines related modules
    - Three high-level modules
    - Ten low-level algorithms
  - Six new algorithms required in this program
    - the main line
    - a menu module to interact with the user
    - a module to get data from the user
    - a module to add a node
    - a module to remove a node
    - a module to print the list

Processing a Linked List

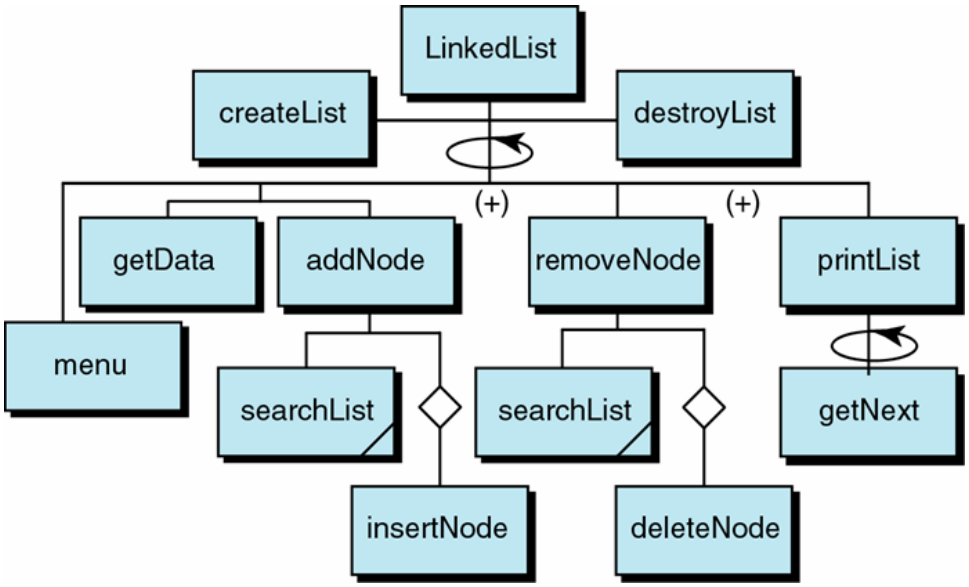


Figure 3-18 Design for build linked list

The Main Line Logic

```
algorithm buildLinkedList
This program builds a linked list that can be modified or printed
by the user.
 1 print (Welcome to exploring linked lists.)
 2 createList (list)
 3 loop (option not quit)
    1   option = menu ()
    2   if (option add)
        1   dataIn = getData()
        2   addNode (list, dataIn)
    3   elseif (option delete)
        1   print (Enter key of data to be deleted.)
        2   read (deleteKey)
        3   removeNode (list, deleteKey)
    4   elseif (option print)
        1   printList (list)
    5   end if
 4 end loop
 5 destroyList (list)
 6 print (Exploration complete. Thank you.)
end buildLinkedList
```

Algorithm 3-11 Build a linked list

The Menu Module

```
algorithm menu
    Display a menu and read user option.
    Pre     Nothing
    Return Valid choice

 1  print (.....MENU.....)
 2  print (A: Add new data.)
 3  print (D: Delete data.)
 4  print (P: Print list.)
 5  print (Q: Quit)
 6  valid = false
 7  loop (valid false)
    1   print (Enter your choice: ")
    2   read (choice)
    3   if (choice equal 'A' or 'D' or 'P' or 'Q')
        1   valid = true
    4   else
        1   print (Invalid choice. Choices are <A, D, P, Q>)
    5   end if
 8  end loop
 9  return choice
end menu
```

Algorithm 3-12 Menu for building a linked list

Add Node

- Steps to add a node
  - Get the data for the new node
  - Search the list to located the insertion point
  - Once we know the insertion point
    - use insert node to physically place the data in the list

```
algorithm addNode (ref list   <metadata>,
                   val dataIn <dataType>)
Add data to a linked list.
    Pre     list is metadata structure to a valid list
            dataIn are data to be inserted into list
    Post    data have been inserted into list in key sequence
```

Algorithm 3-13 Add node to linked list

Add Node

```
1  found = searchList (list, pPre, pLoc, dataIn.key)
2  if (found)
   Error: Key already exists
      1   print (Error: Data already in list. Not added.)
3  else
      1   success = insertNode (list, pPre, dataIn)
      2   if (success false)
          1   print (Error: Out of memory. Program quitting.)
          2   abort algorithm
      3   end if
4  return
end addNode
```

Algorithm 3-13 Add node to linked list

Add Node

- Algorithm 3-13 Analysis
  - Using two previously developed algorithms
    - searchList
    - insertNode
  - When inserting data into a list
    - we need to decide whether duplicate data will be allowed
    - A node is generally considered a duplicate
      - if its key matches an existing node's key
  - Decide how to handle memory overflow
    - abort the program and print a message

Remove Node

- Remove node
  - Given the key of the node to be removed
    - call searchList to determine its location and predecessor
    - then deleteNode to do the physical deletion

```
algorithm removeNode (ref list  <metadata>,
                      val key   <keyType>)
This algorithm deletes a node from the linked list.
    Pre   list is metadata structure to a valid list
          key is the key to be located and deleted
    Post  the node has been deleted
           -or- a warning message printed if not found
```

Algorithm 3-14 Remove data from linked list

Remove Node

```
 1  found = searchList (list, pPre, pLoc, key)
 2  if (found)
    1   deleteNode (list, pPre, pLoc, deleteData)
 3  else
    1   print (Error: Key not in list.)
 4  end if
 5  return
end removeNode
```

Algorithm 3-14 Remove data from linked list

- Algorithm 3-14 analysis
  - Statement 2:
    - Verify that the data being deleted actually exist
    - Return true if the key was found
    - Return false if it doesn't exist

Print List

- Print List
  - One of the more common uses of link list traversals
    - We traverse the list and print all of the key fields
  - In algorithm 3-15,
    - Print a sequential number and the key from the data
    - To do this, we need to call traverse (getNext)
    - Continue a loop until all of the data have been printed

Print List

```
algorithm printList (val list <metadata>)
This algorithm traverses a linked list and prints the key in each
node.
    Pre   list is metadata structure to a valid list
    Post  All keys have been printed

 1  if (emptyList (list))
    1   print (No data in list.)
 2  else
    1   print (**** Begin Data Print ****)
    2   count = 0
    3   moreData = getNext (list, 0, dataPtr)
    4   loop (moreData true)
        1   count = count + 1
        2   print (count, dataPtr->key)
        3   moreData = getNext (list, 1, dataPtr)
    5   end loop
    6   print (**** End Data Print ****)
 3  end if
 4  return
end printList
```

Algorithm 3-15 Print linked list

Print List

- Algorithm 3-15 analysis
  - Test for an empty list before starting the loop
  - Printing a start and end message
    - To assure the user that all of the data were printed
  - The data from each node are numbered
    - So that the user knows how many elements are in the list
  - The first call to traverse (getNext) must
    - have a Fromwhere flag of zero

Testing Insert and Delete Logic

- Testing linked list algorithms
  - Requires careful planning
- Four tests to validate the insert logic
  - Insert a node into a null list
  - Insert a node before the first data node
  - Insert between two data nodes
  - Insert after the last node

Testing Insert and Delete Logic

- Testing the delete logic
  - Delete to a null list
  - Delete the first data node in the list
  - Delete a node between two data nodes
  - Delete the node at the end of the list

3-5 List Applications

- Append lists
  - Build two linked lists
  - Append the second one to the end of the first one
  - How to do that?
    - Change the link pointer in the last node of the first list
      - to point to the first node of the second list
    - See Figure 3-19 in next page

Append Lists

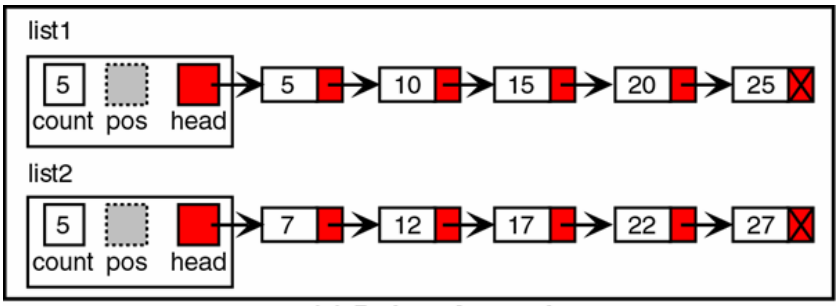
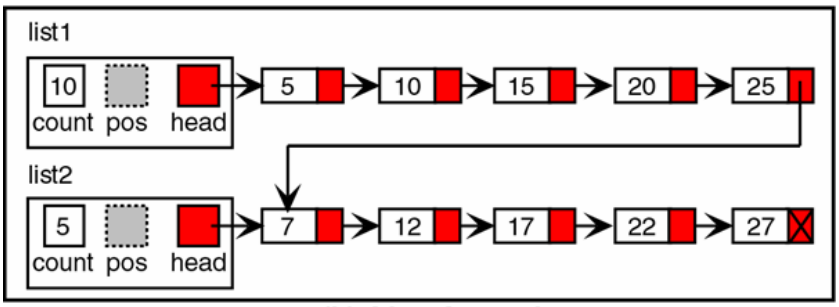


Figure 3-19 Append linked lists (a) Before Append



(b) After Append

Append Lists

```
algorithm appendTwoLists
    Creates two linked lists and then appends the second to
    the first.
 1  print (This program creates two lists and then appends them)
 2  print (Enter first file name)
 3  read  (fileName)
 4  open  (fileName)
 5  build     (list1, fileName)
 6  printList (list1)
 7  print (Enter second file name)
 8  read  (fileName)
 9  open  (fileName)
10  build     (list2, fileName)
11  printList (list2)
12  append    (list1, list2)
    The lists are now appended. Print to prove success.
13  printList (list1)
14  return
end appendTwoLists
```

Algorithm 3-16 Append two linked lists

Append Lists

- Algorithm 3-16 analysis
  - The algorithm's main line just calls to algorithms
    - **build**
    - **append**
  - build and append will be introduced as follow
    - build in algorithm 3-17
    - append in algorithm 3-18

Append Lists

```
algorithm build (ref list <metadata>,
                 val file <data file>)
This algorithm builds a linked list from data in a file.
    Pre  list is metadata structure for list to be built
         File must exist
    Post List has been built

 1  createList (list)
 2  loop (not end of file)
    1   read (file into dataIn)
    2   searchList (list, pPre, pLoc, dataIn.key)
    3   insertNode (list, pPre, dataIn)
 3  end loop
 4  return
end build
```

Algorithm 3-17 Build linked lists

Append Lists

```
algorithm append (ref list1 <metadata>,
                  val list2 <metadata>)
This algorithm appends the second list to the end of the first.
    Pre   both lists are metadata structures to valid list
    Post  Second list appended to the first list
 1  if (list1.count zero)
    1   list1.head = list2.head
 2  else
    1   pLoc = list1.head
    2   loop (pLoc->link not null)
        1   pLoc = pLoc->link
    3   end loop
    4   pLoc->link = list2.head
 3  end if
 4  list1.count = list1.count + list2.count
 5  return
end append
```

Algorithm 3-18 Append linked lists

Append Lists

- Algorithm 3-18 analysis
  - Steps
    - If the first list is null
      - set its head pointer to the head pointer of the second list
    - Traverse the first list to find the last node
    - Change the last node's link pointer
      - to point to the beginning of the second list
    - Add the counts for the two lists
      - Place the sum in the count for the first list

List Applications

- Array of lists
  - The data for each linked list are read from a set of existing files and inserted into the list
  - After the program builds the lists, they are printed to verify that they are complete

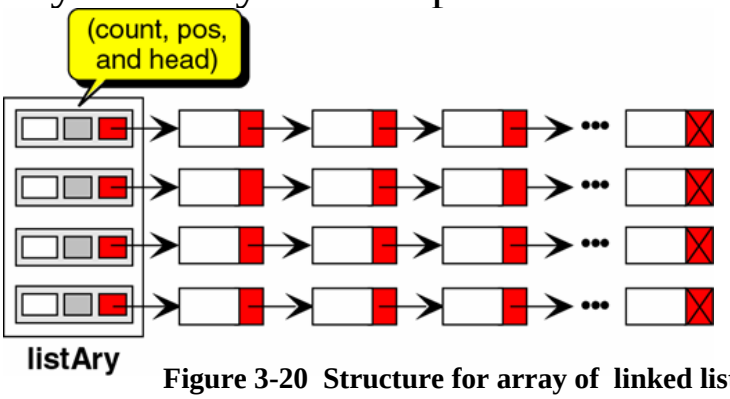


Figure 3-20 Structure for array of linked lists

Array of Lists

- Linked list of a two-dimensional array
  - Each linked list represents one row in the array
  - The nodes in the linked list represent the columns

```
algorithm arrayOfLists
Create an array of linked lists. Each list is read from
a file. After the lists have been created, they are
printed to verify that they were built properly.
 1 print (Begin array of linked lists)
 2 print (How many list do you want?)
 3 read  (numLists)
 4 buildArys (listArray, numLists)
 5 printArys (listArray, numLists)
 6 print (End of array of linked lists)
end arrayOfLists
```

Algorithm 3-19 Array of linked lists

Array of Lists

```
algorithm buildArrays (ref listArray  <metadata>,
                       val numLists   <integer>)
Read files and insert contents into linked lists pointed to by
an array of head pointers.
   Pre   listArray and the number of elements in the array
   Post  Array of linked lists built
 1 row = 0
 2 loop (row < numLists)
   1   print (Enter file name)
   2   read  (fileName)
   3   open  (fileName)
   4   build (listArray[row], fileName)
   5   close (fileName)
   6   row = row + 1
 3 end loop
 4 return
end buildArrays
```

Algorithm 3-20 Build array of linked lists

Array of Lists

```
algorithm printArys (val listArray <metadata>,
                     val numLists  <integer>)
   This algorithm prints the contents of lists in listArray.
   Pre   lists in listArray has been filled
         numLists is the number of list pointers in array
   Post  The lists have been printed, one after the other
 1 row = 0
 2 loop (row < numLists)
   1   printList (listArray [row])
   2   row = row + 1
 3 end loop
 4 return
end printAry
```

Algorithm 3-21 Print array of linked lists

3-6 Complex Linked List Structures

- Circularly linked list
  - The last node's link points to the first node of the list
  - When inserting or deleting the last node
    - update the rear pointer in the header
    - point the link field of the last node to the first node

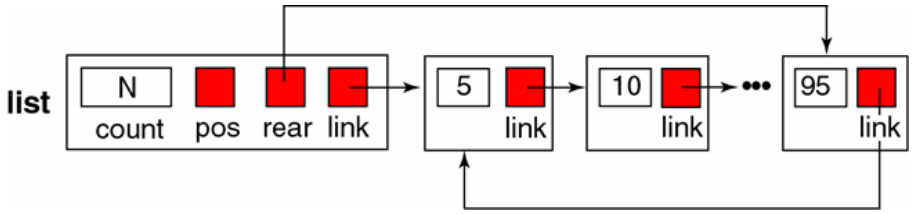


Figure 3-21 A circularly linked list

Circularly Linked Lists

- Allow searching from the middle of the list
  - No difference between beginning and middle in probability
  - However, two problems will occur
  - If target lies before the current node
    - Automatically continue the search from the beginning
  - If target does not exist
    - Save the starting node's address
    - Stop when we have circled around to it

```
loop (target not equal to pLoc->data.key
  AND pLoc->link not equal to startAddress)
```

Complex Linked List Structures

- Doubly linked list
  - One of the most powerful variations of linked lists
  - Each node has two pointers
    - One to its successor
    - The other to its predecessor

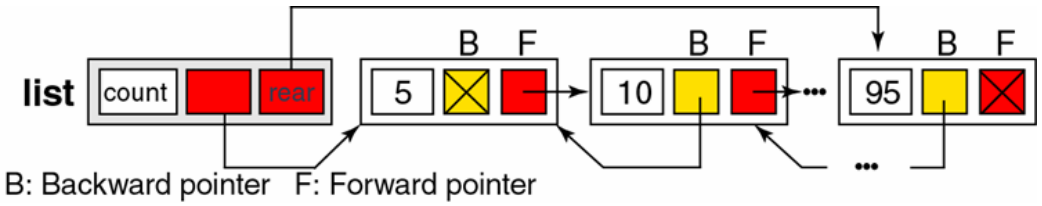


Figure 3-22 A doubly linked list

Doubly Linked Lists

- Three Metadata in the head structure
  - A count
  - A head pointer for traversals
  - A rear pointer
    - It is not required in all doubly linked lists
    - It makes some of list algorithm more efficient

Doubly Linked Lists

- Each node contain two pointer
  - A backward pointer to its predecessor
  - A forward pointer to its successor
- Doubly linked circularly linked list
  - The last forward pointer points to the first node
  - The backward pointer of the first node points to the last node

Doubly Linked Lists

- Insertion in doubly linked list
  - Insert a node into a null list
    - Set the head and rear pointers to point to the new node
    - Set the forward and backward pointers of new node to null
    - Shown in figure 3-23(a)
  - Inserting between two nodes
    - The new node must point to its predecessor and successor
    - The predecessor and successor must point to the new node
    - Shown in figure 3-23(b)

Doubly Linked Lists

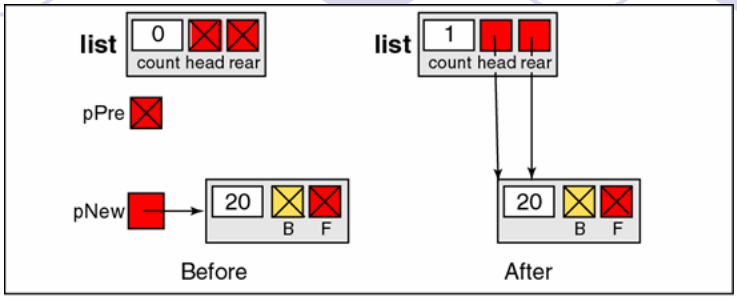
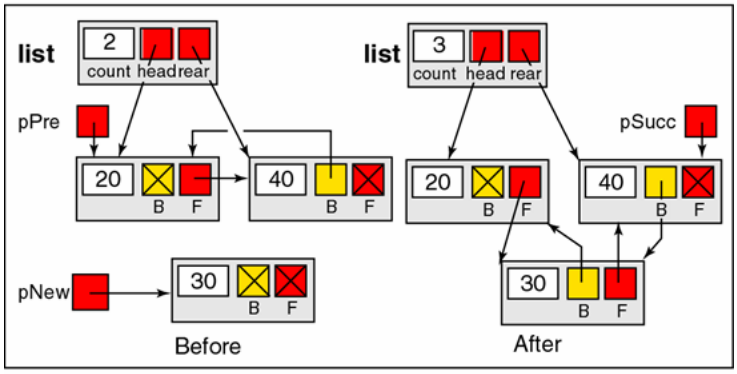


Figure 3-23 Doubly linked list insert (a) Insert into null list or before first node



(b) Insert between two nodes

Doubly Linked Lists

- Inserting at the end of the list
  - New node's back point must point to its predecessor
  - The new node's forward pointer is set to null
    - Because no successor
  - The rear pointer in the head structure must point to the new rear node

Doubly Linked Lists

```
algorithm insertDbl (ref list    <metadata>,
                     val dataIn  <dataType>)
This algorithm inserts data into a doubly linked list.
   Pre    list is metadata structure to a valid list
          dataIn contains the data to be inserted
   Post   The data have been inserted in sequence
   Return <integer> 0: failed--dynamic memory overflow
                    1: successful
                    2: failed--duplicate key presented
 1 if (full list)
   1  return 0
 2 end if
Locate insertion point in list
 3 found = searchList (list, pPre, pSucc, dataIn.key)
 4 if (not found)
   1  allocate (pNew)
    2 pNew->data = dataIn
    3 if (pPre is null)
      Inserting before first node or into empty list
      1  pNew->back  = null
      2  pNew->fore  = list.head
      3  list.head   = pNew
```

Algorithm 3-22 Doubly linked list insert

Doubly Linked Lists

```
    4 else
      Inserting into middle or end of list
      1  pNew->fore  = pPre->fore
      2  pNew->back  = pPre
    5 end if
      Test for insert into null list or at end of list
    6 if (pPre->fore null)
      Inserting at end of list--set rear pointer
      1  list.rear = pNew
    7 else
      Inserting in middle of list--point successor to new
      1  pSucc->back = pNew
    8 end if
    9 pPre->fore = pNew
   10 list.count = list.count + 1
   11 return (1)
 5 end if
   Duplicate data. Key already exists.
 6 return (2)
end insertDbl
```

Algorithm 3-22 Doubly linked list insert *(continued)*

Doubly Linked Lists

- Algorithm 3-22 修正
  - 第4.9行調到4.4.3行
  - 4.6 改成 if (pSucc null)

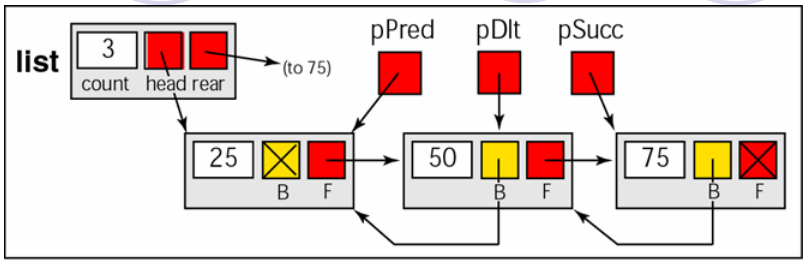
Doubly Linked Lists

- Algorithm 3-22 analysis
  - Return for three different conditions
    - If dynamic memory is full, return 0,
    - If it is successful, then return 1
    - If the insert matches data already in the list, return 2
  - The search provides
    - the location of target's logical predecessor (pPre)
    - and either (pSucc)
      - the location of the node with a key that matches the target
      - the location of the first node with a key greater than target
      - or null if the target is greater than the last node

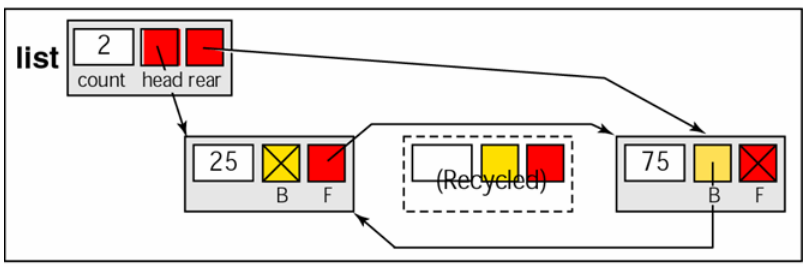
Doubly Linked Lists

- Deletion in doubly linked list
  - Once locate the node to be deleted
    - Change its predecessor's and successor's pointers
    - The predecessor
      - if any, is pointed to the deleted node's successor
    - The successor
      - if any, is set to point to the predecessor
    - Recycle the node

Doubly Linked Lists



(a) Before delete



(b) After deleting 50

Figure 3-24 Doubly linked list delete

Doubly Linked Lists

```
algorithm deleteDbl (ref list  <metadata>,
                     val pDlt  <node pointer>)
This algorithm deletes a node from a doubly linked list.
    Pre    list is metadata structure to a valid list
           pDlt is a pointer to the node to be deleted
    Post   node deleted
 1  if (pDlt null)
    1   abort (Impossible condition in delete double)
 2  end if
 3  list.count  = list.count - 1
 4  if (pDlt->back not null)
        Point predecessor to successor
    1   pPred       = pDlt->back
    2   pPred->fore = pDlt->fore
 5  else
        Update head pointer
    1   list.head = pDlt->fore
 6  end if
```

Algorithm 3-23 Doubly linked list delete

Doubly Linked Lists

```
 7  if (pDlt->fore not null)
        Point successor to predecessor
    1   pSucc       = pDlt->fore
    2   pSucc->back = pDlt->back
 8  else
        Point rear to predecessor
    1   list.rear = pDlt->back
 9  end if
10  recycle (pDlt)
11  return
end deleteDbl
```

Algorithm 3-23 Doubly linked list delete *(continued)*

Doubly Linked Lists

- Algorithm 3-23 analysis
  - The search was done in the calling algorithm
    - We already know the location of the node to be deleted
  - It should never be called with a null delete pointer
    - so we abort the program if we detect one
  - Create the predecessor pointer (Statement 4.1)
  - Create the successor pointer (Statement 7.1)

Complex Linked List Structures

- Multilinked lists
  - A list with two or more logical key sequences
  - For example

President	Year	First Wife
Washington, George	1789	Custis, Martha Dandridge
Adams, John	1797	Smith, Abigail
Jefferson, Thomas	1801	Skelton, Martha Wayles
Madison, James	1809	Todd, Dorothy Payne
Monroe, James	1817	Kortright, Elizabeth
Adams, John Quincy	1825	Johnson, Louisa Catherine
Jackson, Andrew	1829	Robards, Rachel Donelson
Van Buren, Martin	1837	Hoes, Hannah
Harrison, William H	1841	Symmes, Anna
Tyler, John	1841	Christian, Letitia

Table 3-2 First ten presidents of the United States

Multilinked Lists

- The power of Multilinked lists
  - The same set of data can
    - be processed in multiple sequences
  - The data are not replicated
    - The data exist only once, but multiple paths connect the one set of data
  - To process the data in multiple sequences
    - Create a separate set of links for each sequence
    - Each link structure can be **singly linked**, **doubly linked**, or **circularly linked**

Multilinked Lists

- A singly linked multilinked list

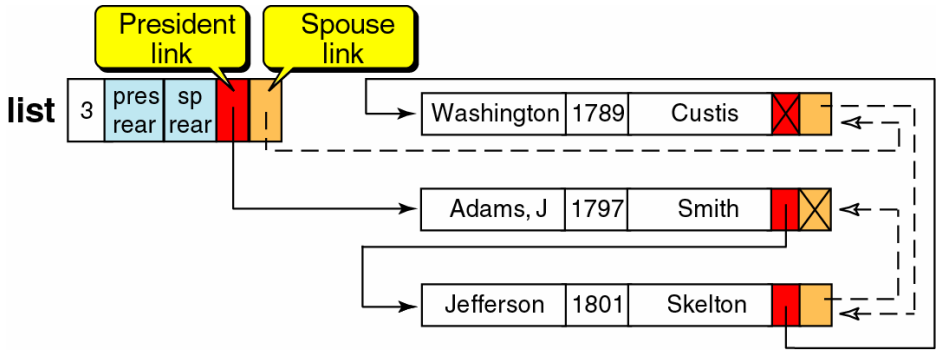


Figure 3-25 Multilinked list of presidents

  - Two logical lists in the one physical list
    - Two link fields
      - one for the president and one for the spouse

Multilinked Lists

- Insert in multilinked lists
  - Algorithm for add node
    - Assume data come from file

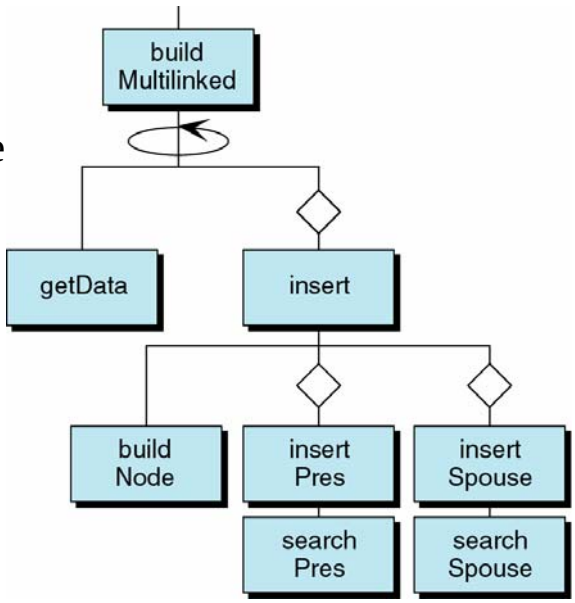


Figure 3-26 Multilinked list add node

Multilinked List Insert

- This build-multilinked module loops again
  - until the entire file has been read and the list build

- Three steps to insert a node
  - Allocating space for the data in buildNode
  - Building the president links in the insert president module
  - Building the spouse links in the insert spouse module

Multilinked List Insert

- Memory allocation is separate from
  - **Searching** and **pointer connections**
  - Because the memory is allocated only once
- The count is added only once
  - because of inserting only one physical node